SQMA Tool

Software Quality & Metrics Analysis

Complete explanation of all 24 features

Why each feature exists | the rules | how every number is calculated
how the code applies it | how the whole website works

VIKNESH KUMAR .S

(SOFTWARE METRICS PROJECT)

Written in simple language so that it can be explained to anyone - step by step.

Contents

Part 1 - What is this website? (in simple words) 3

Part 2 - Words you will meet (mini dictionary) 5

Part 3 - The 24 features, one by one 6

1. Home Dashboard 6
2. Goals (GQM) 7
3. Source Code Upload 7
4. Files 9
5. Code Issues 9
6. Project Metrics 11
7. Bug Management 11
8. Releases 13
9. Control Chart 14
10. Software Size 15
11. Complexity 15
12. Quality Metrics 17
13. Statistical Analysis 18
14. Correlation 19
15. Box Plot 19
16. Decision Tree 21
17. Defect Prediction 22
18. Risk Prediction (Bayesian) 23
19. Security Analysis 24
20. Maintainability 25
21. Alerts 26
22. AI Assistant 27
23. Reports 28
24. Users 28

Part 4 - All thresholds in one table 30

Part 5 - About this project: how I did it, my involvement and research 31

Part 1 - What is this website? (in simple words)

Think of SQMA as a **health check-up centre for software**. When you go to a doctor, the doctor **measures** things (height, weight, blood pressure), **compares** them with normal ranges (rules), **warns** you if something is wrong (alerts), **predicts** future risk, and finally gives you a **report**. SQMA does exactly the same for a software project.

It looks at a project from **two sides**:

- **The code side** - you upload the source code (Python, Java, JavaScript, C, C++). The tool reads every file and measures its size, how complicated it is, how easy it is to maintain, and whether it has dangerous coding habits.
- **The bug side** - you (or your team) record the bugs found in the software. The tool measures how many bugs there are, how old they are, how many escaped to the customer, and whether a release is safe to ship.

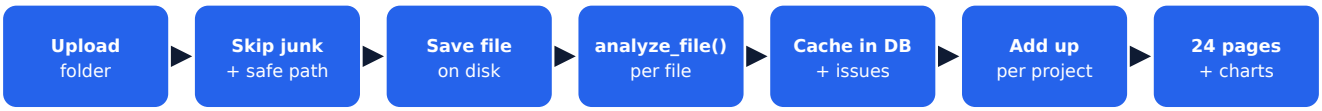
Then it joins both sides together into one dashboard, adds statistics and prediction models, raises alerts, writes a report, and can even ask an AI assistant for advice. Everything except the AI works **without internet**.

1.1 The whole working of the website in 10 steps

Step	What happens	Where in the code
1	**Register / Login.** The first person who registers becomes **Admin**. Passwords are stored as a scrambled hash, never as plain text.	auth.py
2	**Create a project.** A project is one software you want to check. Everything else (files, bugs, releases) belongs to a project.	app.py - create_project()
3	**Upload code.** You choose or drag a folder. The browser removes junk (venv, node_modules, .git, binary files...) and checks the limits (100 MB, 50 files). The server checks again, makes the path safe and saves each file under *uploads/<project>/<path>*.	upload.html (browser JS), file_service.py
4	**Analyse every file.** One function, *analyze_file()*, reads a file and returns its lines, functions, complexity, maintainability and issues. The results are saved (cached) in the database so lists are fast.	analyzers/file_analyzer.py
5	**Record bugs.** Add bugs by hand, load demo bugs, or press *Log as bug* on a code issue. Each bug has severity, status, release, assignee, OWASP category, CVSS score and more.	bugs_views.py, bug_service.py
6	**Combine.** Project numbers are made by adding up the per-file numbers and the bug numbers (e.g. defect density = bugs / KLOC).	analysis_service.py, metrics_summary.py
7	**Show pages.** Each of the 24 pages takes the numbers it needs and draws tables and charts. Analysis pages have an *Analyse: All files / one file* dropdown.	app.py, templates/, static/js/viz.js
8	**Statistics and prediction.** Control chart, box plot, correlation, t-tests, decision tree, random forest, Bayesian risk.	stats_engine/, prediction/, bayesian/
9	**Alerts.** On every visit the tool deletes its old automatic alerts and creates fresh ones from the current numbers.	alerts_engine.py
10	**Report, API and AI.** A printable report with every formula; a REST API for other programs; an optional AI assistant.	reports/, api.py, ai/

1.2 Picture of the data flow

Code side - from the folder you upload to the numbers on screen:



Bug side - from the bug form to alerts and the release light:



1.3 What happens every time you click a link (request life-cycle)

1. The browser sends a request, e.g. */project/1/dashboard*.
2. **Guard (auth.py):** Flask first asks "who is this?" (reads the login from the session). If nobody is logged in, you are sent to the login page. For every form submission (POST) it also checks the secret **CSRF token** so other websites cannot fake your clicks.
3. **Route function** (in app.py or a blueprint file) runs. It reads data from the database with SQLAlchemy (projects, bugs, files, issues).
4. **Calculation modules** turn the data into numbers (metrics/, stats_engine/, prediction/...). Pure Python functions - easy to test.
5. **Template** (templates/*.html, written in Jinja) builds the HTML page. Anything typed by users is escaped so it cannot run as code.
6. In the browser, **viz.js** and Chart.js draw the charts from the numbers embedded in the page. All libraries are stored locally, so no internet is needed.

1.4 Where things live (file map)

File / folder	Job
app.py	Main program: starts Flask, most analysis pages (size, complexity, quality, statistics, prediction, security, maintainability, alerts, report, goals).
auth.py	Login, register, users page, profile and API token, CSRF protection, roles.
bugs_views.py / bug_service.py	Bug Management and Releases pages; one place for bug validation and status rules (used by pages AND the API).
files_views.py / file_service.py	Files, Code Issues, Project Metrics, File detail pages; upload processing, path safety, per-file cache, per-file risk.
analyzers/	Reads source code: file_analyzer (analyze_file), code/complexity/structure/security analyzers (Python), multi_lang (Java/JS/C/C++), python_issues.
metrics/	Formulas: size, defect metrics, maintainability, quality score, CVSS bands, release readiness.
stats_engine/	Descriptive statistics, control chart, box plot, histogram, correlation, hypothesis tests.
prediction/ and bayesian/	Decision tree, random-forest defect prediction, Bayesian risk.
alerts_engine.py	Creates the alerts.
reports/	Builds the report text and the Rule/Formula/Working/Result cards.
api.py	REST API (/api/v1) for other programs.
ai/ and ai_views.py	AI assistant (Google Gemini), optional.

File / folder	Job
config.py	**All** constants and thresholds in one place (limits, bands, readiness rules...).
models.py	Database tables: User, Project, Release, Bug, CodeFile, Issue, Alert, BugComment, ChatMessage...
templates/ and static/	The HTML pages, CSS, and JavaScript (charts, upload, tables).

Part 2 - Words you will meet (mini dictionary)

Word	Simple meaning
LOC / KLOC	Lines Of Code. KLOC = thousand lines (4,800 lines = 4.8 KLOC). Used to compare projects of different size fairly.
Function	A named block of code that does one job (def in Python, method in Java...).
Cyclomatic complexity	How many different paths exist through a function. More if / for / while / and / or = more paths = harder to test and understand.
Maintainability	How easy it is to read, change and fix the code later.
Bug / defect	A mistake in the software, recorded with title, module, severity and status.
Severity	How serious a bug is: Low, Medium, High, Critical.
Open / Closed bug	**Open** = Open, In Progress, Reopened. **Closed** = Resolved, Closed.
Release	A version of the software you ship, e.g. v1.0, v1.1.
Escaped bug	A bug found in **Production** (by customers) instead of in Testing. Fewer is better.
CVSS	A 0-10 score for how dangerous a security weakness is (Low, Medium, High, Critical bands).
OWASP category	A standard list of web-security problem types (SQL Injection, XSS, Broken Authentication...).
Mean / Median	Mean = average. Median = the middle value when sorted.
Standard deviation (sigma)	How spread out the values are around the mean.
Outlier	A value far away from the others.
p-value	In a statistical test: the chance of seeing this result if nothing special is going on. Below 0.05 = result is significant.
Synthetic data	Practice data generated from documented rules, used because a student project has no big history of real defects.

Part 3 - The 24 features, one by one

Every feature below has the same layout: **why it is used**, **what you see**, the **rules**, the **step-by-step calculation**, and **how the code applies it** (file and function names), often with a worked example computed by the real code.

1. Home Dashboard

The one-page summary of the whole project

WHY IT IS USED

A busy person (teacher, manager) wants to know in 10 seconds: "*Is this project in good shape?*" The dashboard answers that by joining code numbers and bug numbers on one page, so nobody has to open 20 pages.

WHAT YOU SEE ON THE PAGE

- A **verdict line** such as *Project condition: Good - 3 of 41 files are High risk (quality 77%, predicted risk Medium) | 8 open / 12 closed bugs | latest release v1.1 is Red*.
- Cards: Quality Score, Predicted Risk, Total Defects + density, Maintainability, LOC, Avg complexity, Files, Security issues.
- Bug-health cards: Open : Closed, average age of open bugs, Escape rate, Release readiness light.
- Charts: bug severity, quality components, release comparison, sub-folder comparison. Tables: module risk (sortable), 10 riskiest files, recent alerts.

RULES

- Quality level: 80+ Excellent, 60-79.99 Good, 40-59.99 Needs Improvement, below 40 Poor (see feature 12).
- 10 riskiest files are ordered by: risk level (Critical first), then lowest maintainability, then highest complexity.
- The alerts box shows the 6 most severe alerts (Critical first).

HOW IT IS CALCULATED (STEP BY STEP)

1. Read all bugs and run *analyze_file()* on every file (aggregated).
2. Compute defect density, maintainability, quality score and predicted risk (features 12, 18, 20).
3. Compute open:closed, average open-bug age, escape rate, per-release rows, per-module rows.
4. Count files whose risk is High or Critical -> "N of M files are High risk".
5. Refresh the alerts (feature 21) and draw everything.

HOW THE CODE APPLIES IT

app.py -> *dashboard()* calls **metrics_summary.project_summary()**, which calls *run_full_code_analysis()* and *compute_scores()* (analysis_service.py) and the bug functions in metrics/defect_metrics.py and metrics/release_metrics.py. *build_verdict()* writes the one-line sentence. The same *project_summary()* also feeds the Report and the REST API, so the three can never disagree.

2. Goals (GQM)

Goal - Question - Metric: why we measure what we measure

WHY IT IS USED

GQM (Basili) is a classic method: don't measure random things. First choose a **Goal**, ask a **Question** about it, then choose **Metrics** that answer the question. This page proves every metric in the tool has a purpose.

WHAT YOU SEE ON THE PAGE

Goal	Question	Metrics shown
Improve reliability	Are defects increasing or under control?	Defect density, reopen rate, total bugs, escape rate
Improve maintainability	Is the code easy to understand and change?	Average complexity, average function size, comment ratio
Improve security	Are there exploitable weaknesses in the code?	Number of High / Medium / Low security findings

HOW IT IS CALCULATED (STEP BY STEP)

Each metric is calculated by the same functions used elsewhere: defect density = bugs / KLOC (feature 7), reopen rate = reopened bugs / total bugs x 100, escape rate (feature 8), comment ratio = comment lines / total lines x 100 (feature 10).

HOW THE CODE APPLIES IT

app.py -> *goals_page()* builds the list *gqm* of goals, each with its question and metric values; **goals.html** shows it.

3. Source Code Upload

How the code gets into the tool - safely

WHY IT IS USED

The tool needs the real source code to measure it. Uploading a **whole folder** (not one file at a time) matches how real projects look, and the safety rules protect your computer from junk, huge files and malicious file names.

WHAT YOU SEE ON THE PAGE

- **Choose Project Folder**, **Choose Files** and a **drag-and-drop** zone (folders are read recursively).
- A **preview**: count, total size, what was skipped and why; a **progress bar** while uploading.
- After upload: the list of analysed files.

RULES

Rule	Value
Total size per upload	100 MB (config.MAX_UPLOAD_BYTES)
Useful files per upload	50 (counted AFTER junk is removed)
Single file	over 2 MB is skipped
Skipped folders	venv, .venv, env, node_modules, .git, __pycache__, build, dist, .idea, .vscode, site-packages, target, bin, obj
Skipped files	binary (known extension such as .png/.exe/.pdf, OR a zero byte in the first 8 KB), minified (*.min.js), unsupported types (.md, .txt ...)
Accepted types	.py .java .js .c .h .cpp .cc .hpp
Same path again	the new file **replaces** the old copy; new paths are added
Unsafe paths	anything with .., an absolute path or a drive letter (C:) is rejected

HOW IT IS CALCULATED (STEP BY STEP)

1. **In the browser** (upload.html): every selected file gets its relative path. The same rules are applied here first, so junk never travels over the network. A dropped folder is read with *webkitGetAsEntry()*; skipped folders are only counted, not opened.
2. The browser shows "Accepted N files, skipped M (venv: .., binary: ..)" and blocks the upload if limits are broken.
3. It sends each file plus a parallel list of paths (one request, with a progress bar).
4. **On the server** the same checks run again (never trust the browser): *sanitize_relpath()* -> *skip_reason()* -> count check.
5. Each accepted file is written to *uploads/<project id>/<relative path>* (sub-folders kept) and analysed (feature 4).
6. If something is over the limit the user gets a friendly sentence ("remove some files or upload folders separately"), not a raw error.

HOW THE CODE APPLIES IT

file_service.py: *sanitize_relpath()* (blocks .., drive letters, odd characters), *skip_reason()* (returns why a file is skipped), *process_upload()* (the whole server-side job), *store_file()* (write to disk, create or replace the database row, call *refresh_file()*). **config.py** holds the limits. **app.py** -> *upload_source()* is the route; it answers JSON to the progress-bar request and a normal page otherwise.

4. Files

Every file, its numbers and its risk colour

WHY IT IS USED

To find the **worst files quickly**. A project-wide average can hide one terrible file; this table lets you sort, search and open that file.

WHAT YOU SEE ON THE PAGE

Table: path, language (a "basic" badge for Java/JS/C/C++), lines, functions, average and max complexity, maintainability with its band, issue count, a **risk chip** (green Low, amber Medium, red High, dark red Critical), plus Re-analyse and Delete buttons. It is sortable and searchable.

RULES

- **Risk of a file** uses the Bayesian model (feature 18) with four yes/no evidences: average complexity > 10; linked-bug density > 3 per KLOC; any security issue; maintainability < 40.
- **Critical** (dark red) = the Bayesian result is High AND (the file has 2 or more High-severity issues OR a function with complexity > 30).

HOW IT IS CALCULATED (STEP BY STEP)

1. When a file is uploaded (or Re-analysed) its numbers are calculated once by *analyze_file()* and stored in columns of the CodeFile table (loc, functions, avg_cc, max_cc, mi_score, issue_count...). That is why the list is fast.
2. The risk chip is worked out when the page opens, because it also depends on how many bugs are linked to the file right now.

HOW THE CODE APPLIES IT

file_service.refresh_file() fills the cached columns and saves the issues. **file_service.file_risk()** turns the evidence into Low/Medium/High/Critical. **file_service.file_rows()** builds the table rows. **files_views.files_page()** shows them.

5. Code Issues

Problems found inside the code - separate from bugs

WHY IT IS USED

A **bug** is a problem someone reported. A **code issue** is a problem the tool found by itself while reading the code (e.g. a dangerous function, a monster function). Keeping them separate avoids flooding the bug list; when you decide to fix an issue you press **Log as bug**.

WHAT YOU SEE ON THE PAGE

Counts per severity, a table (file with folder, line, type, category, severity, description), filters (file, severity, category, type, free text), CSV/JSON export, a **Log as bug** button on each row and **Log all High issues as bugs** (with confirmation).

RULES

Python rule (found with the ast module)	Severity	Category
Function too complex: complexity > 10 (High if > 20)	Medium / High	Complexity
Function too long: > 50 lines (Medium if > 100)	Low / Medium	Maintainability
Too many parameters: > 5 (self/cls not counted)	Low	Maintainability
Deep nesting: blocks nested > 4 levels	Medium	Complexity
Missing docstring (public function or class)	Low	Style
Bare *except:*	Medium	Maintainability
Unused import	Low	Style
Security rules (feature 19): eval, exec, os.system, shell=True, pickle.loads, hard-coded password, SQL string building, assert	High / Medium / Low	Security

Java / JavaScript / C / C++ use the same limits but are found with regular expressions (basic metrics): too complex/long/many parameters/deep nesting, missing comment, empty catch, JS *eval, innerHTML, document.write*, C/C++ *gets, strcpy, sprintf, system*, Java *Runtime.exec* and SQL built by string joining, and hard-coded passwords in every language.

HOW IT IS CALCULATED (STEP BY STEP)

1. During analysis every rule is tested on every function/line; each hit becomes one Issue row (file, line, type, severity, description, suggested fix).
2. **Log as bug** creates a Bug with: title *<type> in <file>:<line>*, description (message, file, line, fix), module = the file's folder, severity from the issue, OWASP category when it is a security issue (SQL concatenation -> SQL Injection, pickle -> Insecure Deserialization, innerHTML -> XSS, hard-coded password -> Sensitive Data Exposure, others -> Other), and the link to the file.
3. The issue remembers the bug number. The button then reads *Logged as bug #12* - so the same issue is never logged twice.

HOW THE CODE APPLIES IT

analyzers/python_issues.py (Python rules), **analyzers/multi_lang.py** (other languages), **config.ISSUE_LIMITS** (the numbers 10/20/50/100/5/4). **file_service.refresh_file()** stores issues and keeps already-logged ones on re-analysis. **files_views.log_issue()** does the Log-as-bug mapping; **issues_export()** writes CSV/JSON (cells starting with = + - @ are neutralised against spreadsheet formula injection).

6. Project Metrics

Totals, per-file table and sub-folder comparison

WHY IT IS USED

Shows the **big picture and the parts**: how the whole project looks, how every file looks, and which **sub-folder** (e.g. backend vs frontend) is healthier.

WHAT YOU SEE ON THE PAGE

- Section 1 - totals: files, LOC, comment %, functions, classes, average/max complexity, maintainability, quality, issues by severity.
- Section 2 - every file with every metric (sortable).
- Section 3 - one row and one grouped bar chart per sub-folder; switch between 1 level and 2 levels deep.

RULES

Files are grouped by their first 1 (or 2) folder levels. Files directly in the root go to a group called **(root)**.

HOW IT IS CALCULATED (STEP BY STEP)

1. For each group: Files = count; LOC = sum; Functions = sum; Issues = sum.
2. Average complexity = mean of the file averages (only files that have functions).
3. Maintainability = **LOC-weighted** average: sum(MI of file x LOC of file) / sum(LOC). A big file counts more than a tiny one.
4. Worst risk = the highest risk chip among the group's files.

HOW THE CODE APPLIES IT

file_service.group_by_folder(rows, depth) makes the groups; **files_views.project_metrics_page()** builds the page; **viz.js -> folderComparison()** draws the grouped bar chart.

7. Bug Management

Record, edit and track bugs

WHY IT IS USED

Code numbers only tell half the story. Real quality also means *how many bugs were found, how serious, how fast fixed, and who is responsible*. This page is the tracker that feeds all bug metrics.

WHAT YOU SEE ON THE PAGE

An add form and a list with filters (status, severity, module, assignee, release, OWASP, search, **My bugs**). Fields: title, description, module, date reported, severity, priority, status, assignee, release, found in (Testing/Production), OWASP category, CVSS score (with a live colour badge), linked source file. Buttons: edit, resolve/reopen, delete, **Ask AI**. Demo-bug and delete-all buttons.

RULES

Rule	Value
Statuses	Open, In Progress, Resolved, Reopened, Closed
Open bug / Closed bug	Open = Open, In Progress, Reopened. Closed = Resolved, Closed
Moving to Resolved/Closed	sets the resolved date
Moving back to an open status	clears the resolved date
Moving to Reopened	adds 1 to the reopen count
CVSS	0.0 to 10.0 only. Bands: 0 None, 0.1-3.9 Low, 4.0-6.9 Medium, 7.0-8.9 High, 9.0-10.0 Critical
Date reported	cannot be in the future
Assignee	must be an existing Developer or Tester
Release / file	must belong to the same project
Title	required, max 250 characters

HOW IT IS CALCULATED (STEP BY STEP)

1. The form (or the API) sends raw text.
2. *validate_bug_data()* checks every field and returns clear messages for wrong values.
3. *create_bug()* / *update_bug()* save it; *apply_status()* keeps resolved date and reopen count consistent.
4. After every change the bug-related alerts are recomputed (feature 21).

HOW THE CODE APPLIES IT

bug_service.py (validation, status rules, conversion to JSON - shared by the pages and the REST API, so both behave the same), **bugs_views.py** (routes), **metrics/cvss.py** (*cvss_band()* - the one place where the bands are defined), **config.py** (status lists, OWASP list).

8. Releases

Which release is safe to ship? - traffic light

WHY IT IS USED

Teams ship **versions**. Grouping bugs by release lets you compare v1.0 vs v1.1 and, most importantly, decide **go / no-go** for a release using fixed, fair rules.

WHAT YOU SEE ON THE PAGE

Add / edit / delete releases (deleting a release only un-tags its bugs, bugs are never deleted). Per release: total, open, closed, Critical+High, escaped bugs, escape rate, and a **green / yellow / red circle** with the reasons written in words.

RULES

Colour	Condition (checked on the OPEN bugs of that release)
RED	any open **Critical** bug, OR **3 or more** open High bugs, OR any open security bug with CVSS ≥ 9
YELLOW	any open High bug, OR more open bugs than closed bugs, OR any open bug older than 30 days
GREEN	none of the above

Red is checked first, then Yellow. A release that turns Red also creates a Critical alert. Thresholds are in **config.READINESS**.

HOW IT IS CALCULATED (STEP BY STEP)

1. Take the bugs whose release is this release.
2. Count open Critical, open High, open security bugs with CVSS ≥ 9, open bugs older than 30 days, open vs closed.
3. Apply the rules top-down: Red -> Yellow -> Green. Collect the reasons (e.g. "1 open Critical bug; 2 open bugs older than 30 days").
4. **Escape rate** of the release = bugs with Found-in = Production / bugs in the release x 100 (0 when there are no bugs).

WORKED EXAMPLE

v1.1 has 20 bugs: 3 found in Production -> escape rate 3/20 x 100 = **15 %**. It has 1 open Critical bug -> the light is **RED**, reason "1 open Critical bug".

HOW THE CODE APPLIES IT

metrics/release_metrics.py: *release_readiness()* (the traffic light), *release_table()* (rows), *latest_release()*, *comparison_data()* (chart). **metrics/defect_metrics.py**: *escape_rate()*. **bugs_views.py**: *releases()*, *release_edit()*, *release_delete()*.

9. Control Chart

Is the bug arrival rate stable, or is something unusual happening?

WHY IT IS USED

From Statistical Process Control. If bugs arrive at about the same rate every day the process is **stable**; a sudden spike (a "bad day") means something went wrong (a bad deployment, a risky change). The chart makes the spike visible and gives a fair rule to call it "unusual".

WHAT YOU SEE ON THE PAGE

A line chart of bugs reported per day (or per week), a green centre line (CL), red dashed upper (UCL) and lower (LCL) limits, and red dots for points outside the limits. Selectors: variance type (Sample / Population) and sigma (1, 2 or 3; default 3), remembered in your session and printed on the Report.

RULES

- Data = number of bugs created on each day (days with zero bugs count as 0). If the bug dates span more than 120 days it switches to weeks (Monday start).
- Needs at least 2 points.
- CL = mean. UCL = mean + k x sigma. LCL = max(0, mean - k x sigma) - a bug count cannot be negative.
- A point above UCL or below LCL is "out of control" (unusual variation).
- **Sample** sigma divides by n-1 (use when the data is a sample of a bigger process - default for bug data). **Population** sigma divides by n (you have every value).
- k = 3 catches only very unusual points (about 99.7 % of normal variation); k = 2 is more sensitive but gives more false alarms.

WORKED EXAMPLE

Daily bug counts: 2, 3, 1, 4, 10. Mean = (2+3+1+4+10)/5 = **4**. Deviations: -2, -1, -3, 0, +6 -> squares 4, 1, 9, 0, 36 -> sum 50.

Sample: variance = 50/4 = 12.5, sigma = **3.536**, UCL(3) = 4 + 3 x 3.536 = **14.607**, LCL = max(0, 4 - 10.6) = **0**.

Population: variance = 50/5 = 10, sigma = **3.162**, UCL(3) = **13.487**. With 2-sigma (sample) UCL = **11.071**.

The 10 is below UCL 14.6, so at 3-sigma the process is called stable; with 2-sigma (UCL 11.07) it is still inside. Any day above 14.6 bugs (e.g. 15) would be flagged as unusual.

HOW THE CODE APPLIES IT

stats_engine/control_chart.py -> *build_control_chart(values, labels, sigma_mult, variance_type)*. **stats_engine/descriptive.py** -> *spread()* picks *statistics.variance* (n-1) or *pvariance* (n). **app.py** -> *bug_series()* counts bugs per day/week; *stat_options()* reads and remembers the options. **viz.js** draws the chart.

10. Software Size

How big is the code?

WHY IT IS USED

Size is the base of many other metrics (defect density divides by size). It also shows how much documentation (comments) the code has and how big the average function is - big functions are harder to maintain.

WHAT YOU SEE ON THE PAGE

Total files, total lines, logical (real code) lines, comment lines, blank lines, comment ratio, number of functions, classes, imports, average function size, and charts.

RULES

- Total lines = every line in the file. Blank = empty after trimming. Comment = line that starts with # (Python) or is only a comment (other languages).
- **Logical lines = total - blank - comment.**
- Function size = number of lines from its first line to its last line.
- **Average function size** = sum of function sizes / number of functions. **Comment ratio** = comment lines / total lines x 100.

WORKED EXAMPLE

A file has 120 lines, 20 blank, 18 comments -> logical = 120 - 20 - 18 = **82**. Comment ratio = 18/120 x 100 = **15 %**. Functions of 10, 20 and 30 lines -> average = **20 lines**.

HOW THE CODE APPLIES IT

analyzers/code_analyzer.py -> *analyze_source()* parses a Python file with *ast* (functions, classes, imports, line spans). Other languages: *multi_lang.analyze_multi()* strips comments and strings first, then counts. **metrics/size_metrics.py** -> *aggregate_size()* adds the files up.

11. Complexity

How complicated is each function? (cyclomatic complexity)

WHY IT IS USED

Complicated functions hide bugs, need more test cases and are hard to change. McCabe's cyclomatic complexity gives a number: the **count of independent paths** through the code. It tells you **which functions to simplify first**.

WHAT YOU SEE ON THE PAGE

A table of functions ranked by complexity with a risk level, average and maximum complexity, and charts.

RULES

Complexity V(G) = 1 + number of decision points. Decision points counted in Python: *if / elif, for, async for, while, except, assert*, every extra operand of *and / or*, and every *if* inside a comprehension. (Java/JS/C/C++: if, else if, for, while, case, catch, &&, ||, ?:).

Complexity	Level	Meaning
1 - 5	Low	simple, low risk
6 - 10	Moderate	acceptable
11 - 20	High	complex - refactor candidate
21 and more	Very High	very complex - high risk

These bands are project-defined thresholds, stated as such on the page.

WORKED EXAMPLE

`def f(a, b):` has one *if a and b* (1 for the if + 1 for the "and") and one *for* loop: complexity = 1 + 1 + 1 + 1 = **4** (Low).

HOW THE CODE APPLIES IT

analyzers/complexity_analyzer.py. A visitor walks the syntax tree of each function and adds 1 at every decision point:

```
class _ComplexityVisitor(ast.NodeVisitor):
    def __init__(self): self.score = 1          # the base path
    def visit_If(self, node):  self.score += 1; self.generic_visit(node)
    def visit_BoolOp(self, node):               # a and b and c -> +2
        self.score += max(0, len(node.values) - 1); self.generic_visit(node)
```

12. Quality Metrics

One overall quality score from four parts

WHY IT IS USED

Management wants **one number**. The quality score blends maintainability, complexity, defects and security into a single 0-100 score and a word (Excellent... Poor).

WHAT YOU SEE ON THE PAGE

The overall score and level, the four component scores as bars, the formula and the inputs used.

RULES

Quality Score = 0.30 x Maintainability + 0.25 x Complexity + 0.25 x Defects + 0.20 x Security (project-defined weights; each part is first changed to a 0-100 scale).

Component	How it becomes 0-100
Maintainability	the Maintainability Index itself (feature 20)
Complexity	100 - (average complexity - 1) x (100/29), kept between 0 and 100 (1 -> 100 points, 30 or more -> 0)
Defects	100 - defect density x 5, between 0 and 100 (0 bugs/KLOC -> 100, 20 or more -> 0)
Security	100 - (15 x High findings + 7 x Medium + 2 x Low), between 0 and 100

Score	Level
80 - 100	Excellent
60 - 79.99	Good
40 - 59.99	Needs Improvement
below 40	Poor

WORKED EXAMPLE

Maintainability 58.09 (feature 20), average complexity 4, defect density 2.5 per KLOC, security: 1 High + 1 Medium finding.

Complexity part = 100 - (4-1) x 3.448 = **89.66**. Defect part = 100 - 2.5 x 5 = **87.5**. Security part = 100 - (15 + 7) = **78**.

Score = 0.30 x 58.09 + 0.25 x 89.66 + 0.25 x 87.5 + 0.20 x 78 = **77.32** -> **Good**.

HOW THE CODE APPLIES IT

```
# metrics/quality_metrics.py
total = 0.30*m + 0.25*c + 0.25*d + 0.20*s
if total >= 80: level = 'Excellent'
elif total >= 60: level = 'Good'
elif total >= 40: level = 'Needs Improvement'
else: level = 'Poor'
```

13. Statistical Analysis

Descriptive statistics, histogram and hypothesis tests

WHY IT IS USED

Averages alone can mislead. Statistics show **spread** (are all functions similar or are some extreme?) and let us **test claims** scientifically (e.g. "do serious bugs really take longer to fix?") instead of guessing.

WHAT YOU SEE ON THE PAGE

Two blocks (function complexity and function size) with count, mean, median, variance, standard deviation, min, max, coefficient of variation; a histogram for each; and a **Hypothesis tests** section with four tests. Selectors: variance type and target complexity.

RULES

- **Mean** = sum / n. **Median** = middle value. **Sample variance** = sum(x-mean)^2 / (n-1); **population variance** = sum(x-mean)^2 / n; std dev = square root of variance.
- **Coefficient of variation** CV = std dev / mean x 100: below 15 % low variation, below 35 % moderate, otherwise high.
- **Histogram** (Sturges' rule): number of bins k = ceil(1 + log2 n); class width = ceil((max - min + 1)/k); each value goes into exactly one bin.
- **Hypothesis tests**, always with alpha = 0.05: if p < 0.05 we *reject H0*, otherwise *fail to reject H0*.

HOW IT IS CALCULATED (STEP BY STEP)

Test	H0 (nothing special)	What is compared
Welch two-sample t-test	High/Critical bugs and Low/Medium bugs take the same mean time to resolve	resolution time in hours of resolved bugs (needs 2+ per group)
One-sample t-test	Mean function complexity = target (default 10, editable)	all function complexities vs the target
Chi-square independence (module x severity)	module and severity are independent	counts table of bugs
Chi-square independence (release x severity)	release and severity are independent	counts table of bugs

Each test shows H0/H1, the data table, the statistic, degrees of freedom, p-value, alpha and a plain-English conclusion. If the sample is too small (fewer than 2 values per group, or chi-square cells with expected count below 5) a **warning** is shown instead of a crash.

WORKED EXAMPLE

Data 2, 4, 4, 4, 5, 5, 7, 9: mean = **5**, median = **4.5**, sum of squared deviations = 32.

Sample variance = 32/7 = **4.571**, std dev = **2.138**. Population variance = 32/8 = **4**, std dev = **2.0**. CV (sample) = 2.138/5 x 100 = **42.76 %** -> high variation.

Histogram of 1,2,2,3,3,3,4,5,8,9 (n = 10): k = ceil(1 + log2 10) = 5, width = ceil(9/5) = 2 -> bins 1-2, 3-4, 5-6, 7-8, 9-10 with counts **[3, 4, 1, 1, 1]**.

HOW THE CODE APPLIES IT

stats_engine/descriptive.py (*describe()*, *spread()*), **stats_engine/histogram.py** (*build_histogram()*), **stats_engine/hypothesis.py** (*welch_resolution_time()*, *one_sample_complexity()*, *chi_square_independence()*) using **scipy.stats** (*ttest_ind(equal_var=False)*, *ttest_1samp*, *chi2_contingency*). Route: **app.py -> statistics_page()**.

14. Correlation

Do bigger functions tend to be more complex?

WHY IT IS USED

To check a common belief with data: "*long functions are complicated functions*". If true, splitting long functions is a good way to reduce complexity.

WHAT YOU SEE ON THE PAGE

A scatter plot (each dot = one function: size on X, complexity on Y), a trend line, the Pearson r value, its strength label and a plain sentence.

RULES

Pearson r = sum((x-mx)(y-my)) / sqrt(sum((x-mx)^2) x sum((y-my)^2)), always between -1 and +1.

|r|	Strength
0.7 or more	Strong
0.4 - 0.69	Moderate
0.2 - 0.39	Weak
below 0.2	Very weak / none

Positive r: when one goes up the other goes up. Negative r: one goes up, the other goes down. Needs at least 2 pairs; if all values are identical the tool says correlation cannot be computed. The trend line is the least-squares line: slope b = sum((x-mx)(y-my)) / sum((x-mx)^2), intercept a = my - b x mx. r squared tells how much of the change is explained.

WORKED EXAMPLE

Function sizes 5, 10, 20, 40 and complexities 1, 3, 6, 12 -> r = **0.999** (Strong, positive), slope = 0.3096, r squared = 0.998.

HOW THE CODE APPLIES IT

stats_engine/correlation.py -> *pearson_correlation()*. **app.py** -> *correlation_page()* feeds it the size and complexity of every function; **viz.js** draws the scatter and trend line.

15. Box Plot

See the spread and the outliers at a glance

WHY IT IS USED

A box plot shows where the **middle 50 %** of the values lie and highlights **outliers** - e.g. one function with complexity 40 among functions with complexity 3-6. Outliers are the first places to refactor.

WHAT YOU SEE ON THE PAGE

A horizontal box with median line, whiskers and red outlier dots; a table with min, Q1, median, Q3, max, IQR and fences. You can switch between function complexity and function size.

RULES

- Q1, median, Q3 = 25th, 50th, 75th percentile (linear interpolation). **IQR = Q3 - Q1**.
- **Lower fence = Q1 - 1.5 x IQR; upper fence = Q3 + 1.5 x IQR.**
- Outlier = a value outside the fences. Whisker ends = the smallest and largest values that are NOT outliers.

WORKED EXAMPLE

Values 2, 3, 3, 4, 5, 6, 7, 30: Q1 = **3.0**, median = **4.5**, Q3 = **6.25**, IQR = 3.25. Upper fence = 6.25 + 1.5 x 3.25 = **11.125** -> the value **30** is an outlier; the whisker ends at 7.

HOW THE CODE APPLIES IT

stats_engine/boxplot.py -> *build_boxplot()* with the helper *_percentile()*. Drawn by **viz.js**.

16. Decision Tree

An explainable yes/no flowchart that predicts risk

WHY IT IS USED

A decision tree explains **why** a prediction was made, as a chain of simple questions ("Is complexity <= 9.4?"). It is easy to defend: you can follow the exact path your project took.

WHAT YOU SEE ON THE PAGE

The whole tree as a flowchart, the path followed by **this project** highlighted, the predicted risk (Low / Medium / High), probabilities and feature importance.

RULES

Inputs (5 features from the project): total **LOC**, average **complexity**, **defect density**, **reopen rate** (% of bugs ever reopened), number of **security issues**.

Training labels come from this scoring rule (project-defined): add points - complexity > 10: 1 point (> 5: 0.5); defect density > 8: 1 (> 3: 0.5); reopen rate > 20: 1 (> 8: 0.5); security issues > 2: 1 (> 0: 0.5); LOC > 800: 0.5. **Total 2.5 or more = High; 1.0 or more = Medium; else Low.**

HOW IT IS CALCULATED (STEP BY STEP)

1. Generate 450 synthetic training rows from three project profiles (healthy, moderate, unhealthy) and label them with the rule above.
2. Train a scikit-learn *DecisionTreeClassifier(max_depth=4)* so the tree stays small and readable.
3. Feed in this project's five numbers and walk down the tree to a leaf = the prediction.
4. Convert the tree to nested data and draw it as a flowchart, highlighting the visited nodes.

WORKED EXAMPLE

LOC 900 (+0.5), complexity 12 (+1), defect density 5 (+0.5), reopen rate 10 (+0.5), 1 security issue (+0.5) -> score 3.0 -> label **High**.

NOTE

Honest limitation (written on the page too): there is no big real defect history in a student project, so the model learns from **synthetic data made from documented rules**. It demonstrates the technique and is explainable; with real history you would replace the generator.

HOW THE CODE APPLIES IT

prediction/decision_tree.py: *_label_from_rules()*, *_generate_training_data()*, *train_and_predict()*, *_serialize_tree()* (builds the flowchart data and the path).

17. Defect Prediction

How likely is it that this project has defects? (Random Forest)

WHY IT IS USED

A single tree can be unstable. A **random forest** asks many trees and averages them, giving a more reliable **probability** (0-100 %) of defects, not just a label.

WHAT YOU SEE ON THE PAGE

Defect probability %, risk level, feature importance chart, the number of training rows in each class, the **hold-out accuracy**, and the note that training data is synthetic.

RULES

- Features (5): loc, complexity, defect_density, reopen_rate, security_issues. (The old duplicate "module size" feature was removed - it was the same as loc.)
- Training data: 500 synthetic rows, generated in a **stratified** way so both classes (defect / no defect) get 50 %. Model: 80 trees, max depth 6, *class_weight*="*balanced*".
- **Hold-out accuracy**: 25 % of the rows are kept hidden, the model is trained on the other 75 %, then scored on the hidden rows. The final model is retrained on all rows.
- Risk level: probability 66 % or more = High, 33 % or more = Medium, otherwise Low.

HOW IT IS CALCULATED (STEP BY STEP)

1. The generator draws random project profiles and computes a defect probability with a logistic formula from the five features; the label is drawn from it, and a row is accepted only while its class still has room (so the split is 50/50).
2. Train/test split, fit, measure accuracy on the test part.
3. Predict for this project: the forest returns the average of its trees' class probabilities = defect probability.

HOW THE CODE APPLIES IT

prediction/defect_prediction.py -> *_generate_training_data()*, *predict_defect_risk()* (uses *RandomForestClassifier*, *train_test_split(stratify=y)*, *accuracy_score*).

18. Risk Prediction (Bayesian)

Risk probability from evidence - every step visible

WHY IT IS USED

A Naive Bayes calculation is written out by hand (no black box) so that **every probability can be explained**. It answers: "Given the evidence I see in the project, how likely is the risk to be High, Medium or Low?"

WHAT YOU SEE ON THE PAGE

Probability bars for High / Medium / Low, the predicted class, which evidence was true or false, and the step-by-step multiplication for each class.

RULES

Evidence (yes/no)	Counts as TRUE when
high_complexity	average complexity > 10
has_defect_history	defect density > 3 per KLOC
security_findings	at least 1 security issue found
low_maintainability	maintainability index < 40

Class	Prior	P(high complexity)	P(defect history)	P(security)	P(low maint.)
High	0.25	0.75	0.70	0.65	0.60
Medium	0.35	0.40	0.45	0.30	0.35
Low	0.40	0.10	0.15	0.05	0.10

Priors and likelihoods are documented, project-defined estimates (not from a big dataset) - the page says so.

HOW IT IS CALCULATED (STEP BY STEP)

1. For each class start with its prior probability.
2. For each of the 4 evidences multiply by P(evidence | class) if the evidence is true, or by 1 - P(...) if false.
3. Add the three results and divide each by the total (normalise) so they sum to 1.
4. The class with the biggest probability is the prediction.

WORKED EXAMPLE

Evidence: high complexity = TRUE, defect history = FALSE, security findings = TRUE, low maintainability = FALSE.

High = 0.25 x 0.75 x (1-0.70) x 0.65 x (1-0.60) = 0.014625. **Medium** = 0.35 x 0.40 x (1-0.45) x 0.30 x (1-0.35)... (for Medium: 0.35 x 0.40 x 0.55 x 0.30 x 0.65) = 0.015015. **Low** = 0.40 x 0.10 x 0.85 x 0.05 x 0.90 = 0.00153.

Total = 0.031170 -> High = **0.469**, Medium = **0.482**, Low = **0.049** -> predicted **Medium** (High and Medium are almost tied - the page shows the probabilities, not just the winner).

HOW THE CODE APPLIES IT

bayesian/risk_model.py -> *compute_risk_probability()* (PRIOR and LIKELIHOOD_TRUE tables). **prediction/risk_prediction.py** -> *evaluate_risk()* turns the numbers into the four evidences using THRESHOLDS. The same model gives the risk chip of each file (feature 4).

19. Security Analysis

Finds dangerous coding habits (rule-based scanner)

WHY IT IS USED

Security bugs are the most expensive. A simple scanner can catch the classic mistakes early, and each finding maps to one readable rule so it can be explained to anyone.

WHAT YOU SEE ON THE PAGE

Counts of High / Medium / Low findings, a padlock diagram, a chart by type, a table of every finding (file, line, type, description) and the list of rules used.

RULES

Python rule	Severity	Why it is dangerous
Use of eval()	High	runs any text as code (code injection)
Use of exec()	High	runs any code (code injection)
subprocess with shell=True	High	shell injection if input is not cleaned
pickle.loads	High	unpickling untrusted data can run code
SQL string concatenation	High	SQL injection - use parameterised queries
os.system call	Medium	command injection
Hard-coded password / secret / token	Medium	credentials in source code leak
assert used for security	Low	assert is removed when Python runs with -O

Other languages: JS eval / innerHTML / document.write; C/C++ gets / strcpy / sprintf / system; Java Runtime.exec, SQL built with +, empty catch; hard-coded secrets everywhere.

HOW IT IS CALCULATED (STEP BY STEP)

1. Python: the code is parsed into a syntax tree; calls such as *eval(...)* are found exactly with their line number. Text rules (regular expressions) find passwords and SQL joins line by line.
2. Every finding becomes one Issue (category Security) and counts into High/Medium/Low totals.
3. **Effect on quality:** the security part of the quality score falls by 15 per High, 7 per Medium and 2 per Low finding (feature 12), and any finding turns the Bayesian "security_findings" evidence on.

NOTE

Honest limitation: this is a light, explainable course-project scanner, **not** a replacement for professional tools such as Bandit, Semgrep or SonarQube.

HOW THE CODE APPLIES IT

analyzers/security_analyzer.py -> *analyze_security()* (AST checks + regex checks, list *RULES*). **analysis_service.security_summary()** prepares the counts and the diagram data.

20. Maintainability

How easy will it be to change this code later?

WHY IT IS USED

Code is read and changed far more often than written. The **Maintainability Index (MI)** is a classic score (0-100) that predicts how costly future changes will be. Low MI = expensive to maintain.

WHAT YOU SEE ON THE PAGE

The project index with its band, the interpretation bands, the formula, and a per-file table (lowest first) with average function lines, average complexity, MI and band.

RULES

MI = max(0, min(100, (171 - 5.2 x ln(HV) - 0.23 x CC - 16.2 x ln(avgFnLOC)) x 100 / 171))

- **avgFnLOC** = average function size of the file (lines). **CC** = average cyclomatic complexity of the file.
- **HV** (Halstead volume) is approximated as avgFnLOC x log2(12) - a documented approximation, not a full Halstead count.
- A file with no functions is treated as one unit (its logical lines, CC = 1).
- **Project MI = LOC-weighted average of the file MIs** (a 500-line file counts 5 times a 100-line file).

MI	Band
55 - 100	Highly maintainable
40 - 54.9	Moderately maintainable
below 40	Difficult to maintain

WORKED EXAMPLE

Average function 20 lines, CC 4: HV = 20 x 3.585 = 71.7; 171 - 5.2 x ln(71.7) - 0.23 x 4 - 16.2 x ln(20) = 99.33; MI = 99.33 x 100/171 = **58.09** -> Highly maintainable.

50 lines, CC 25 -> raw 74.89 -> MI **43.8** (Moderate). 100 lines, CC 30 -> raw 58.91 -> MI **34.45** (Difficult).

Two files: 100 LOC with MI 60 and 300 LOC with MI 40 -> project MI = (100 x 60 + 300 x 40) / 400 = **45** (Moderate).

NOTE

Earlier versions used total project LOC, which made almost every real project look "difficult". Using the **average function size per file** and bands 55/40 fixes that (a clean small project is Highly maintainable, a messy one Difficult).

HOW THE CODE APPLIES IT

```
# metrics/maintainability.py
hv  = fn_loc * math.log2(12)
raw = 171 - 5.2*math.log(hv) - 0.23*cc - 16.2*math.log(fn_loc)
score = max(0, min(100, raw * 100 / 171))
# project value: sum(mi*loc) / sum(loc)   (project_maintainability)
```

21. Alerts

The tool warns you before you have to look

WHY IT IS USED

Nobody checks 24 pages every day. Alerts **push** the important problems to you: old bugs, a growing backlog, dangerous security bugs, a release that is not ready, and bad code numbers.

WHAT YOU SEE ON THE PAGE

A list sorted Critical -> Warning -> Info with message, level and source (bug / code / manual), a dismiss button, the rule summary, and the **Get AI recommendations** block with rule-based tips.

RULES

Alert	Condition	Level
Aging bug	a bug still open after more than 30 days (one alert per bug)	Warning; Critical if the bug is Critical/High
Growing backlog	open bugs > closed bugs	Warning; Critical if open > 2 x closed
Critical security bug	open bug with an OWASP category AND (severity Critical/High OR CVSS ≥ 7.0)	Critical
Release is Red	release readiness is Red (feature 8)	Critical
Defect density	> 4 per KLOC / > 8 per KLOC	Warning / Critical
Average complexity	> 10 / > 15	Warning / Critical
Security (code)	one or more High security findings in the code	Critical
Maintainability	MI below 40	Warning

HOW IT IS CALCULATED (STEP BY STEP)

1. On every dashboard / alerts visit (and after every bug change) the project's old automatic alerts are **deleted** (source = bug or code).
2. All rules are checked again against the current data and new alerts are created.
3. Manual alerts are never touched. Result: numbers never go stale and never duplicate.
4. **Dismiss** removes an alert; an automatic alert comes back while its condition is still true.

HOW THE CODE APPLIES IT

alerts_engine.py -> *recompute_bug_alerts()* and *recompute_code_alerts()*. Thresholds in **config.py** (AGING_BUG_DAYS = 30, CRITICAL_CVSS = 7.0). **app.py** -> *refresh_alerts()*, *alerts_page()*. Rule-based tips: **reports/report_generator.py** -> *build_recommendations()*.

22. AI Assistant

Ask questions about your project in plain English

WHY IT IS USED

Numbers are powerful but a person asks questions like "*Which module is riskiest?*" or "*What should we fix first?*". The assistant reads the project summary and answers using the real values.

WHAT YOU SEE ON THE PAGE

A chat page with clickable question chips and saved history, an **Ask AI** button on every bug (likely root cause, suggested fix, how to test, how to prevent; with **Save as comment**), and **Get AI recommendations** on the Alerts and Report pages. Each answer says which engine replied.

HOW IT IS CALCULATED (STEP BY STEP)

1. You ask a question. The server builds a **compact summary** of the project (totals, quality, risk, maintainability, bugs by status/severity/module/release, escape rate, bug age, 10 riskiest files, alerts).
2. **Built-in assistant (default, offline):** the question is matched to a topic by keywords (module, file, release, fix first, bugs, security...) and answered directly from the summary with the real numbers. No key, no internet.
3. **Google Gemini (optional):** if a free key is saved, the summary and your last 12 messages are sent to Gemini and its answer is shown.
4. **Automatic fallback:** if Gemini fails (no internet, quota 429, timeout, empty answer) the built-in assistant answers and a note explains why - the page never shows an error.
5. Answers are stored in the database per project and user.

RULES

- The key can be saved by an Admin on the AI page (written to a local **.env** file, active immediately) or set with the environment variable **GEMINI_API_KEY**. It is never sent to the browser.
- Ask AI on a bug offline picks a category from the OWASP field and the wording (SQL injection, XSS, secrets, errors, complexity...) and gives the four sections.

HOW THE CODE APPLIES IT

ai/local_assistant.py (*answer()*, *prioritized_actions()*, *local_bug_triage()*), **ai/provider.py** (*GeminiProvider.generate()*, urllib only; failures become *AIError*), **ai/context.py** (*build_project_context()*), **ai/settings.py** (.env read/write, key validation), **ai_views.py** (routes and the Gemini-then-fallback logic), **static/js/ai.js**.

23. Reports

A printable report that shows every formula and its working

WHY IT IS USED

For a viva, a review or a submission you must **prove** how each number was obtained. The report puts the rule, the formula, the working with your real numbers and the result for every metric on one printable page.

WHAT YOU SEE ON THE PAGE

Summary, code metrics, defect metrics with charts (including release comparison), statistics, prediction models, security, recommendations, and a **card for each metric** with four lines: **Rule - Formula - Working - Result**. It prints which options were used (sigma, variance type) and documents thresholds (release readiness, CVSS bands, MI bands).

HOW IT IS CALCULATED (STEP BY STEP)

1. The route gathers the same project summary used by the Dashboard.
2. *build_calculations()* creates one card per metric with your actual numbers filled in the working line.
3. *build_report()* adds the text summary and the rule-based recommendations.
4. The browser's Print command makes a clean PDF.

HOW THE CODE APPLIES IT

app.py -> *report_page()*; **reports/report_generator.py** -> *build_report()*, *build_calculations()*, *build_recommendations()*; template **report.html**.

24. Users

Who can log in, and what each role may do

WHY IT IS USED

A tool shared by a team needs identities: to know **who** reported or fixed a bug, to **assign** bugs, and to stop strangers from changing data.

WHAT YOU SEE ON THE PAGE

A list of users with role and number of open bugs assigned; Admins also see add / edit / delete. Your own **Profile** shows your API token and lets you change your password.

RULES

Role	Can do
Admin	everything: add, edit, delete users and change roles; delete a project. The first registered user is Admin. The last Admin cannot be removed.
Developer	use all project pages, add/edit bugs, be assigned bugs (default role for new registrations)
Tester	same as Developer; used for people who report and verify bugs

- Passwords: at least 8 characters, stored only as a hash (werkzeug.security).
- Every page except login/register needs a login.
- Every form has a hidden CSRF token that is checked on submit.
- The REST API accepts the session or *Authorization: Bearer <token>*; wrong or missing token gives a JSON 401.

HOW THE CODE APPLIES IT

auth.py: *login()*, *register()*, *load_user_and_guard()* (login + CSRF check before every request), *admin_required*, *users()*, *user_edit()*, *profile()*. **api.py**: *api_auth()* decorator. Model: **models.py -> User**.

Part 4 - All thresholds in one table

Topic	Rule / value	Where it is set
Upload limits	100 MB per upload, 50 useful files, 2 MB per file	config.py
Complexity levels	1-5 Low, 6-10 Moderate, 11-20 High, 21+ Very High	analyzers/complexity_analyzer.py
Issue limits	complex > 10 (very > 20); long > 50 (very > 100); params > 5; nesting > 4	config.ISSUE_LIMITS
Maintainability bands	55+ high, 40-54.9 moderate, below 40 difficult	config.MI_BANDS
Quality weights	0.30 maintainability, 0.25 complexity, 0.25 defects, 0.20 security	metrics/quality_metrics.py
Quality levels	80+ Excellent, 60+ Good, 40+ Needs Improvement, else Poor	metrics/quality_metrics.py
Security penalty	15 per High, 7 per Medium, 2 per Low	metrics/quality_metrics.py
CVSS bands	0 None, 0.1-3.9 Low, 4-6.9 Medium, 7-8.9 High, 9-10 Critical	metrics/cvss.py
Aging bug	open for more than 30 days	config.AGING_BUG_DAYS
Critical security bug	CVSS ≥ 7.0 or severity Critical/High (with an OWASP category)	config.CRITICAL_CVSS, alerts_engine.py
Release readiness	Red: open Critical / 3+ open High / security CVSS ≥ 9. Yellow: open High / open > closed / old bug	config.READINESS
Code alerts	defect density 4 / 8; avg complexity 10 / 15; High security finding; MI < 40	alerts_engine.py
Bayesian evidence	complexity > 10; defect density > 3; any security issue; MI < 40	prediction/risk_prediction.py
Defect-prediction levels	66 % High, 33 % Medium	prediction/defect_prediction.py
Correlation strength	0.7 strong, 0.4 moderate, 0.2 weak	stats_engine/correlation.py
Box-plot fences	Q1 - 1.5 IQR, Q3 + 1.5 IQR	stats_engine/boxplot.py
Control chart	k = 1, 2 or 3 sigma (default 3); sample or population	config.py, stats_engine/control_chart.py
Hypothesis tests	alpha = 0.05; target complexity default 10	config.ALPHA, stats_engine/hypothesis.py

Part 5 - About this project: how I did it, my involvement and research

5.1 The project at a glance

Item	Details
Project name	SQMA - Software Quality & Metrics Analysis Tool
Done by	**VIKNESH KUMAR .S** (Software Metrics project)
Type	Web application that measures software quality from source code and from bug tracking
Technology	Python, Flask, SQLAlchemy, SQLite, scikit-learn, SciPy, NumPy, Bootstrap, Chart.js, JavaScript
Size	24 features (pages), about 5,900 lines of Python and 2,300 lines of page templates, 241 automated test checks (all passing)
Languages analysed	Python (full analysis), Java, JavaScript, C and C++ (basic metrics)
Runs	Locally in the browser on Python 3.13, works offline (only the optional Google Gemini AI needs internet)

5.2 How I did this project (step by step)

1. **Idea and problem.** Software quality is usually judged by feel. I wanted one tool that measures it with standard formulas from the *Software Metrics* subject, for both the code and the bugs, and that can explain every number it shows.
2. **Requirements.** I wrote a detailed master requirements document with **36 requirements** in seven areas: bug management (assignee, releases, OWASP, CVSS), missing metrics and dashboard parts, alerts, AI features, fixes, a REST API, and folder upload with per-file analysis. I also fixed the key decisions in advance: real login with roles, SQLite, a free Google Gemini key, upload limits (100 MB, 50 files, 2 MB), supported languages and Python 3.13.
3. **Research.** I studied the standards and methods behind each feature (see 5.4) so that every formula and threshold has a source and a reason.
4. **Design.** I planned the layers (browser, Flask blueprints, services, analysis engines, database), the database tables and their relations, and kept every threshold in one configuration file so rules can be changed in one place.
5. **Build in three phases.** Phase 1: login, users, bug management, releases, alerts, statistics tests, fixes and the REST API. Phase 2: folder upload, per-file analysis, multi-language analysers, code issues, project metrics and the file detail page. Phase 3: the AI assistant and the project documents. I built it with AI assistance (Claude by Anthropic) for writing the code, while I directed the work phase by phase.
6. **Testing.** After each phase I ran the application, opened every page in the browser, tried the features with demo data and a sample project folder, and reported problems. Automated tests (241 checks) cover login, bugs, releases, alerts, the API, uploads, per-file analysis and the AI features.
7. **Improvement.** Testing led to real fixes: the maintainability bands were re-calibrated, the duplicate feature in defect prediction was removed, a path bug that wrongly handled the *.git* folder was fixed, and the AI page was changed to work offline with a built-in assistant when no key is available.
8. **Documentation and sharing.** I wrote the README, this explanation document, the report/PPT/demo-video material, and published the project to GitHub.

5.3 My involvement

- **Planning and decisions:** I defined what the tool must do, chose the technology and decided the rules (limits, bands, thresholds, roles, supported languages).

- **Directing the development:** I gave the requirements one phase at a time and checked each phase before moving to the next one.
- **Running and testing:** I installed the project, ran it in VS Code and the browser, tested the pages (upload, dashboard, control chart, goals, AI assistant, and more) and reported what did not work.
- **Understanding every feature:** I studied the formulas, rules and code behind all 24 features so that I can explain how each number is calculated and where it is implemented.
- **Presentation:** I prepared the explanations, this document and the repository description so the project can be demonstrated and defended.

5.4 Research I did for this project

Topic studied	What I learned and used
Cyclomatic complexity (McCabe, 1976)	A function's complexity is 1 plus its decision points; bands Low, Moderate, High, Very High.
Maintainability Index (Oman & Hagemeister; Coleman)	A 0-100 score from size, complexity and Halstead volume. I found that using total project size gave unfair results, so I calculate it per file and combine files by lines of code.
Defect metrics	Defect density per KLOC, escape rate, reopen rate, mean time to repair and bug age.
CVSS v3 and the OWASP Top 10	How to score and classify security weaknesses; used for bug fields, alerts and the Red release rule.
GQM (Goal-Question-Metric, Basili)	How to link every metric to a goal and a question (the Goals page).
Statistical process control	Shewhart control charts with 1, 2 or 3 sigma limits; sample versus population variance.
Descriptive and inferential statistics	Box plot and the 1.5 x IQR rule, histogram (Sturges), Pearson correlation, Welch and one-sample t-tests, chi-square tests, p-value and alpha = 0.05.
Machine learning basics	Decision tree (explainable), random forest (probability), Naive Bayes (step-by-step risk), train/test split and hold-out accuracy, why synthetic data is a limitation.
Secure web development	Password hashing, CSRF tokens, safe file paths for uploads, escaping output, token-based API, keeping API keys on the server.
Tools and compatibility	Which library versions work on Python 3.13, how to use the free Google Gemini API, and how to analyse code with Python's *ast* module.

5.5 Who is this project useful to?

User	How it helps
Students and teachers	A working demonstration of software metrics, statistics and prediction; every formula is shown with its working, which suits viva and teaching.
Developers	Find the most complex, least maintainable and most risky files; see dangerous patterns such as eval() or hard-coded passwords; turn issues into tracked bugs.
Testers and QA engineers	Record bugs with severity, OWASP category and CVSS; track open, closed and escaped bugs; see the control chart of bug arrival.
Project managers and team leads	One dashboard with a verdict line, release readiness (Red/Yellow/Green), alerts and a printable report for go/no-go decisions.
Small teams and start-ups	A free, offline tool that needs no paid service; works on any computer with Python.
Security reviewers	A quick rule-based scan and a list of open security bugs by OWASP category and CVSS.
Researchers	A base to experiment with software-quality metrics, using the REST API and exportable issue lists.

5.6 What I gained from this project

- A clear understanding of how real software-quality measures are defined, calculated and interpreted.
- Practical skills in Python, Flask, databases, REST APIs, JavaScript charts, statistics and machine-learning basics.
- Experience in planning a large project in phases, testing it, finding problems and improving the design.
- Awareness of the limits of metrics and models: numbers support decisions, they do not replace judgement.

5.7 Limitations and future work

Limitations: Java, JavaScript, C and C++ use pattern-based analysis (not a full parser); the prediction models are trained on synthetic data derived from documented rules; the security scanner is a light, explainable rule set; SQLite and the Flask development server suit single-team use. **Future work:** real parsers (tree-sitter) for more languages, training on real defect history, Git history (change frequency), a CI pipeline hook through the REST API, trends over time, PDF export, and a production database and server.

5.8 Conclusion

SQMA joins code metrics, bug metrics, statistics, prediction and alerts in one tool that explains every number with its rule, formula and working. It can show the whole project or a single file, tells clearly whether a release is safe to ship, and runs on a normal computer without internet. The project shows how the concepts of *Software Metrics* can be turned into a practical tool that helps people build better software.

> **NOTE**
> **How to run it:** open the *sqma* folder in VS Code, run *python -m venv venv*, activate it, *pip install -r requirements.txt*, *python app.py*, then open http://127.0.0.1:5000 and register (the first account is Admin).

VIKNESH KUMAR .S - Software Metrics Project